

13.2-4

Show that any arbitrary n -node binary search tree can be transformed into any other arbitrary n -node binary search tree using $O(n)$ rotations. (*Hint*: First show that at most $n - 1$ right rotations suffice to transform the tree into a right-going chain.)

★ 13.2-5

We say that a binary search tree T_1 can be *right-converted* to binary search tree T_2 if it is possible to obtain T_2 from T_1 via a series of calls to RIGHT-ROTATE. Give an example of two trees T_1 and T_2 such that T_1 cannot be right-converted to T_2 . Then, show that if a tree T_1 can be right-converted to T_2 , it can be right-converted using $O(n^2)$ calls to RIGHT-ROTATE.

13.3 Insertion

In order to insert a node into a red-black tree with n internal nodes in $O(\lg n)$ time and maintain the red-black properties, we'll need to slightly modify the TREE-INSERT procedure on page 321. The procedure RB-INSERT starts by inserting node z into the tree T as if it were an ordinary binary search tree, and then it colors z red. (Exercise 13.3-1 asks you to explain why to make node z red rather than black.) To guarantee that the red-black properties are preserved, an auxiliary procedure RB-INSERT-FIXUP on the facing page recolors nodes and performs rotations. The call RB-INSERT(T, z) inserts node z , whose *key* is assumed to have already been filled in, into the red-black tree T .

RB-INSERT(T, z)

```
1  $x = T.root$                 // node being compared with  $z$ 
2  $y = T.nil$                   //  $y$  will be parent of  $z$ 
3 while  $x \neq T.nil$           // descend until reaching the sentinel
4    $y = x$ 
5   if  $z.key < x.key$ 
6      $x = x.left$ 
```

```

7   else  $x = x.right$ 
8  $z.p = y$                                 // found the location—insert  $z$  with parent  $y$ 
9 if  $y == T.nil$ 
10    $T.root = z$                             // tree  $T$  was empty
11 elseif  $z.key < y.key$ 
12    $y.left = z$ 
13 else  $y.right = z$ 
14  $z.left = T.nil$                           // both of  $z$ 's children are the sentinel
15  $z.right = T.nil$ 
16  $z.color = RED$                           // the new node starts out red
17 RB-INSERT-FIXUP( $T, z$ )                  // correct any violations of red-black
     $z$ )                                     properties

```

The procedures TREE-INSERT and RB-INSERT differ in four ways. First, all instances of NIL in TREE-INSERT are replaced by *T.nil*. Second, lines 14–15 of RB-INSERT set *z.left* and *z.right* to *T.nil*, in order to maintain the proper tree structure. (TREE-INSERT assumed that *z*'s children were already NIL.) Third, line 16 colors *z* red. Fourth, because coloring *z* red may cause a violation of one of the red-black properties, line 17 of RB-INSERT calls RB-INSERT-FIXUP(*T*, *z*) in order to restore the red-black properties.

RB-INSERT-FIXUP(*T*, *z*)

```

1 while  $z.p.color == RED$ 
2   if  $z.p == z.p.p.left$                     // is  $z$ 's parent a left child?
3      $y = z.p.p.right$                       //  $y$  is  $z$ 's uncle
4     if  $y.color == RED$                    // are  $z$ 's parent and uncle both
                                           red?
5        $z.p.color = BLACK$ 
6        $y.color = BLACK$ 
7        $z.p.p.color = RED$ 
8        $z = z.p.p$ 
9   else
10     if  $z == z.p.right$ 
11        $z = z.p$ 

```

} case 1

```

12         LEFT-ROTATE(T, z)
13         z.p.color = BLACK
14         z.p.p.color = RED
15         RIGHT-ROTATE(T,
16                     z.p.p)
17     else // same as lines 3–15, but with “right” and “left” exchanged
18         y = z.p.p.left
19         if y.color == RED
20             z.p.color = BLACK
21             y.color = BLACK
22             z.p.p.color = RED
23             z = z.p.p
24         else
25             if z == z.p.left
26                 z = z.p
27                 RIGHT-ROTATE(T,
28                             z)
29                 z.p.color = BLACK
30                 z.p.p.color = RED
31                 LEFT-ROTATE(T, z.p.p)
32     T.root.color = BLACK

```

To understand how RB-INSERT-FIXUP works, let’s examine the code in three major steps. First, we’ll determine which violations of the red-black properties might arise in RB-INSERT upon inserting node *z* and coloring it red. Second, we’ll consider the overall goal of the **while** loop in lines 1–29. Finally, we’ll explore each of the three cases within the **while** loop’s body (case 2 falls through into case 3, so these two cases are not mutually exclusive) and see how they accomplish the goal.

In describing the structure of a red-black tree, we’ll often need to refer to the sibling of a node’s parent. We use the term *uncle* for such a node.¹ Figure 13.4 shows how RB-INSERT-FIXUP operates on a sample red-black tree, with cases depending in part on the colors of a node, its parent, and its uncle.

What violations of the red-black properties might occur upon the call to RB-INSERT-FIXUP? Property 1 certainly continues to hold (every node is either red or black), as does property 3 (every leaf is black), since both children of the newly inserted red node are the sentinel *T.nil*. Property 5, which says that the number of black nodes is the same on every simple path from a given node, is satisfied as well, because node *z* replaces the (black) sentinel, and node *z* is red with sentinel children. Thus, the only properties that might be violated are property 2, which requires the root to be black, and property 4, which says that a red node cannot have a red child. Both possible violations may arise because *z* is colored red. Property 2 is violated if *z* is the root, and property 4 is violated if *z*'s parent is red. Figure 13.4(a) shows a violation of property 4 after the node *z* has been inserted.

The **while** loop of lines 1–29 has two symmetric possibilities: lines 3–15 deal with the situation in which node *z*'s parent *z.p* is a left child of *z*'s grandparent *z.p.p*, and lines 17–29 apply when *z*'s parent is a right child. Our proof will focus only on lines 3–15, relying on the symmetry in lines 17–29.

We'll show that the **while** loop maintains the following three-part invariant at the start of each iteration of the loop:

- a. Node *z* is red.
- b. If *z.p* is the root, then *z.p* is black.
- c. If the tree violates any of the red-black properties, then it violates at most one of them, and the violation is of either property 2 or property 4, but not both. If the tree violates property 2, it is because *z* is the root and is red. If the tree violates property 4, it is because both *z* and *z.p* are red.

Part (c), which deals with violations of red-black properties, is more central to showing that RB-INSERT-FIXUP restores the red-black properties than parts (a) and (b), which we'll use along the way to understand situations in the code. Because we'll be focusing on node *z* and nodes near it in the tree, it helps to know from part (a) that *z* is red. Part (b) will help show that *z*'s grandparent *z.p.p* exists when it's

referenced in lines 2, 3, 7, 8, 14, and 15 (recall that we're focusing only on lines 3–15).

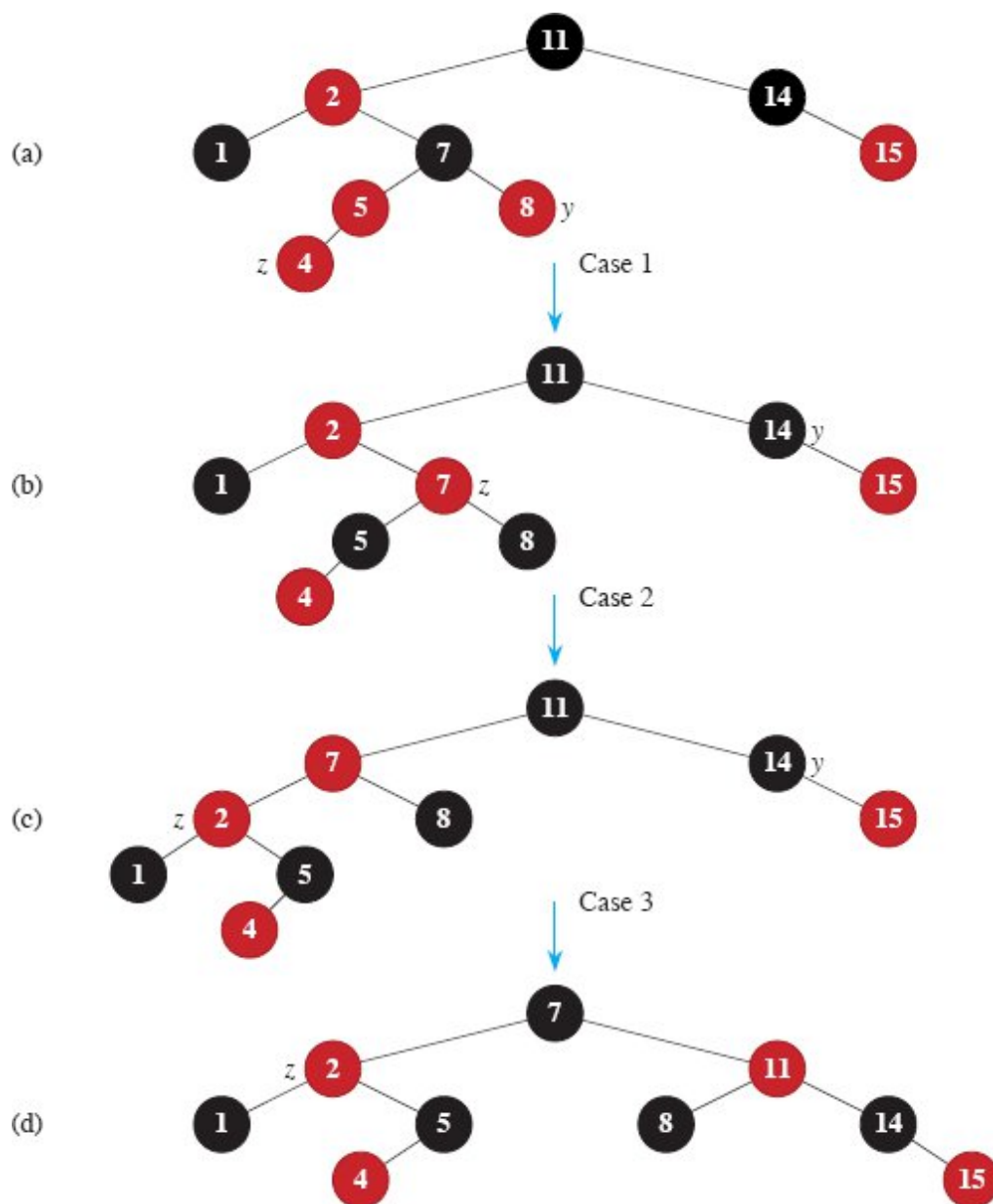


Figure 13.4 The operation of RB-INSERT-FIXUP. **(a)** A node z after insertion. Because both z and its parent $z.p$ are red, a violation of property 4 occurs. Since z 's uncle y is red, case 1 in the code applies. Node z 's grandparent $z.p.p$ must be black, and its blackness transfers down one level to z 's parent and uncle. Once the pointer z moves up two levels in the tree, the tree shown in **(b)** results. Once again, z and its parent are both red, but this time z 's uncle y is black. Since z is the right child of $z.p$, case 2 applies. Performing a left rotation results in the tree in **(c)**. Now z is the left child of its parent, and case 3 applies. Recoloring and right rotation yield the tree in **(d)**, which is a legal red-black tree.

Recall that to use a loop invariant, we need to show that the invariant is true upon entering the first iteration of the loop, that each iteration maintains it, that the loop terminates, and that the loop invariant gives us a useful property at loop termination. We'll see that each iteration of the loop has two possible outcomes: either the pointer z moves up the tree, or some rotations occur and then the loop terminates.

Initialization: Before RB-INSERT is called, the red-black tree has no violations. RB-INSERT adds a red node z and calls RB-INSERT-FIXUP. We'll show that each part of the invariant holds at the time RB-INSERT-FIXUP is called:

- a. When RB-INSERT-FIXUP is called, z is the red node that was added.
- b. If $z.p$ is the root, then $z.p$ started out black and did not change before the call of RB-INSERT-FIXUP.
- c. We have already seen that properties 1, 3, and 5 hold when RB-INSERTFIXUP is called.

If the tree violates property 2 (the root must be black), then the red root must be the newly added node z , which is the only internal node in the tree. Because the parent and both children of z are the sentinel, which is black, the tree does not also violate property 4 (both children of a red node are black). Thus this violation of property 2 is the only violation of red-black properties in the entire tree.

If the tree violates property 4, then, because the children of node z are black sentinels and the tree had no other violations prior to z being added, the violation must be because both z and $z.p$ are red. Moreover, the tree violates no other red-black properties.

Maintenance: There are six cases within the **while** loop, but we'll examine only the three cases in lines 3–15, when node z 's parent $z.p$ is a left child of z 's grandparent $z.p.p$. The proof for lines 17–29 is symmetric. The node $z.p.p$ exists, since by part (b) of the loop invariant, if $z.p$ is the root, then $z.p$ is black. Since RB-INSERT-FIXUP enters a loop

iteration only if $z.p$ is red, we know that $z.p$ cannot be the root. Hence, $z.p.p$ exists.

Case 1 differs from cases 2 and 3 by the color of z 's uncle y . Line 3 makes y point to z 's uncle $z.p.p.right$, and line 4 tests y 's color. If y is red, then case 1 executes. Otherwise, control passes to cases 2 and 3. In all three cases, z 's grandparent $z.p.p$ is black, since its parent $z.p$ is red, and property 4 is violated only between z and $z.p$.

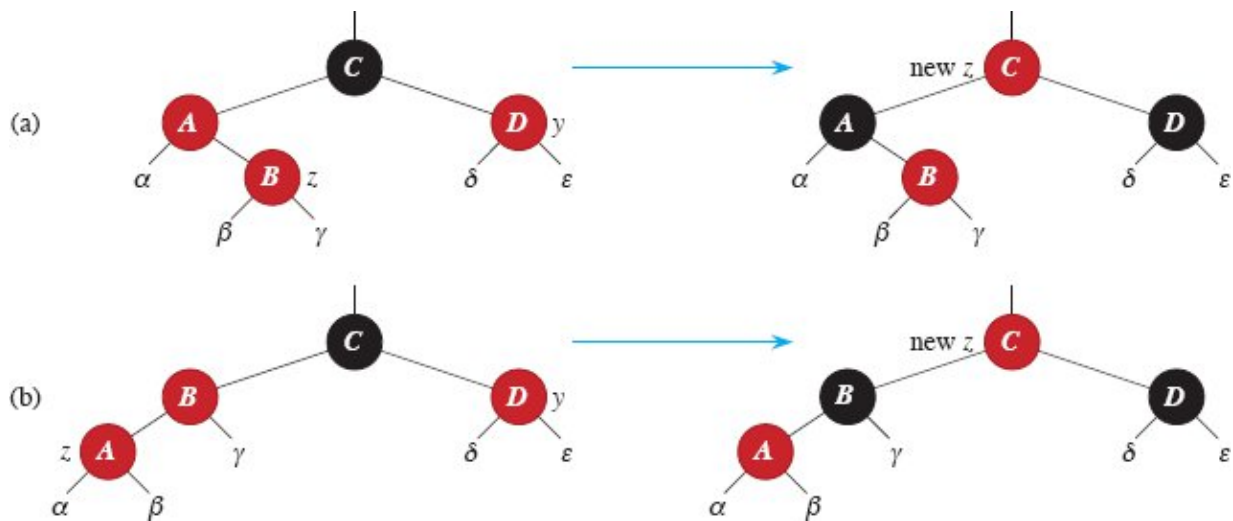


Figure 13.5 Case 1 of the procedure RB-INSERT-FIXUP. Both z and its parent $z.p$ are red, violating property 4. In case 1, z 's uncle y is red. The same action occurs regardless of whether (a) z is a right child or (b) z is a left child. Each of the subtrees α , β , γ , δ , and ϵ has a black root—possibly the sentinel—and each has the same black-height. The code for case 1 moves the blackness of z 's grandparent down to z 's parent and uncle, preserving property 5: all downward simple paths from a node to a leaf have the same number of blacks. The **while** loop continues with node z 's grandparent $z.p.p$ as the new z . If the action of case 1 causes a new violation of property 4 to occur, it must be only between the new z , which is red, and its parent, if it is red as well.

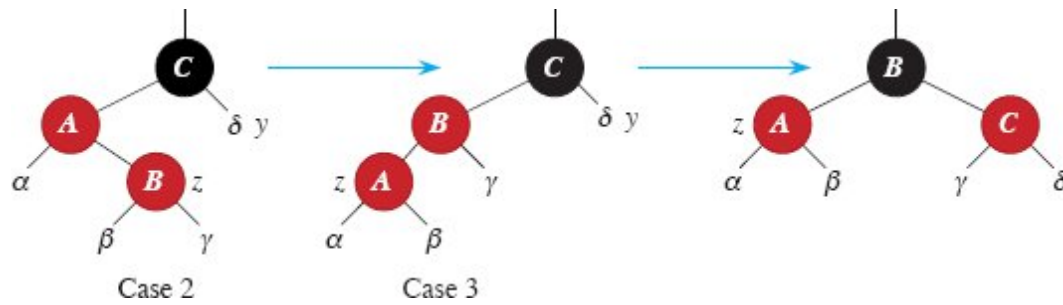


Figure 13.6 Cases 2 and 3 of the procedure RB-INSERT-FIXUP. As in case 1, property 4 is violated in either case 2 or case 3 because z and its parent $z.p$ are both red. Each of the subtrees α , β , γ , and δ has a black root (α , β , and γ from property 4, and δ because otherwise case 1 would apply), and each has the same black-height. Case 2 transforms into case 3 by a left rotation, which preserves property 5: all downward simple paths from a node to a leaf have the same number of blacks. Case 3 causes some color changes and a right rotation, which also preserve property 5. The **while** loop then terminates, because property 4 is satisfied: there are no longer two red nodes in a row.

Case 1. z 's uncle y is red

Figure 13.5 shows the situation for case 1 (lines 5–8), which occurs when both $z.p$ and y are red. Because z 's grandparent $z.p.p$ is black, its blackness can transfer down one level to both $z.p$ and y , thereby fixing the problem of z and $z.p$ both being red. Having had its blackness transferred down one level, z 's grandparent becomes red, thereby maintaining property 5. The **while** loop repeats with $z.p.p$ as the new node z , so that the pointer z moves up two levels in the tree.

Now, we show that case 1 maintains the loop invariant at the start of the next iteration. We use z to denote node z in the current iteration, and $z' = z.p.p$ to denote the node that will be called node z at the test in line 1 upon the next iteration.

- Because this iteration colors $z.p.p$ red, node z' is red at the start of the next iteration.
- The node $z'.p$ is $z.p.p.p$ in this iteration, and the color of this node does not change. If this node is the root, it was black prior to this iteration, and it remains black at the start of the next iteration.

- c. We have already argued that case 1 maintains property 5, and it does not introduce a violation of properties 1 or 3.

If node z' is the root at the start of the next iteration, then case 1 corrected the lone violation of property 4 in this iteration. Since z' is red and it is the root, property 2 becomes the only one that is violated, and this violation is due to z' .

If node z' is not the root at the start of the next iteration, then case 1 has not created a violation of property 2. Case 1 corrected the lone violation of property 4 that existed at the start of this iteration. It then made z' red and left $z'.p$ alone. If $z'.p$ was black, there is no violation of property 4. If $z'.p$ was red, coloring z' red created one violation of property 4, between z' and $z'.p$.

Case 2. z 's uncle y is black and z is a right child

Case 3. z 's uncle y is black and z is a left child

In cases 2 and 3, the color of z 's uncle y is black. We distinguish the two cases, which assume that z 's parent $z.p$ is red and a left child, according to whether z is a right or left child of $z.p$. Lines 11–12 constitute case 2, which is shown in Figure 13.6 together with case 3. In case 2, node z is a right child of its parent. A left rotation immediately transforms the situation into case 3 (lines 13–15), in which node z is a left child. Because both z and $z.p$ are red, the rotation affects neither the black-heights of nodes nor property 5. Whether case 3 executes directly or through case 2, z 's uncle y is black, since otherwise case 1 would have run. Additionally, the node $z.p.p$ exists, since we have argued that this node existed at the time that lines 2 and 3 were executed, and after moving z up one level in line 11 and then down one level in line 12, the identity of $z.p.p$ remains unchanged. Case 3 performs some color changes and a right rotation, which preserve property 5. At this point, there are no longer two red nodes in a row. The **while** loop terminates upon the next test in line 1, since $z.p$ is now black.

We now show that cases 2 and 3 maintain the loop invariant. (As we have just argued, $z.p$ will be black upon the next test in line 1, and the loop body will not execute again.)

- a. Case 2 makes z point to $z.p$, which is red. No further change to z or its color occurs in cases 2 and 3.
- b. Case 3 makes $z.p$ black, so that if $z.p$ is the root at the start of the next iteration, it is black.
- c. As in case 1, properties 1, 3, and 5 are maintained in cases 2 and 3.

Since node z is not the root in cases 2 and 3, we know that there is no violation of property 2. Cases 2 and 3 do not introduce a violation of property 2, since the only node that is made red becomes a child of a black node by the rotation in case 3.

Cases 2 and 3 correct the lone violation of property 4, and they do not introduce another violation.

Termination: To see that the loop terminates, observe that if only case 1 occurs, then the node pointer z moves toward the root in each iteration, so that eventually $z.p$ is black. (If z is the root, then $z.p$ is the sentinel $T.nil$, which is black.) If either case 2 or case 3 occurs, then we've seen that the loop terminates. Since the loop terminates because $z.p$ is black, the tree does not violate property 4 at loop termination. By the loop invariant, the only property that might fail to hold is property 2. Line 30 restores this property by coloring the root black, so that when **RB-INSERT-FIXUP** terminates, all the red-black properties hold.

Thus, we have shown that **RB-INSERT-FIXUP** correctly restores the red-black properties.

Analysis

What is the running time of **RB-INSERT**? Since the height of a red-black tree on n nodes is $O(\lg n)$, lines 1–16 of **RB-INSERT** take $O(\lg n)$

time. In RB-INSERTFIXUP, the **while** loop repeats only if case 1 occurs, and then the pointer z moves two levels up the tree. The total number of times the **while** loop can be executed is therefore $O(\lg n)$. Thus, RB-INSERT takes a total of $O(\lg n)$ time. Moreover, it never performs more than two rotations, since the **while** loop terminates if case 2 or case 3 is executed.

Exercises

13.3-1

Line 16 of RB-INSERT sets the color of the newly inserted node z to red. If instead z 's color were set to black, then property 4 of a red-black tree would not be violated. Why not set z 's color to black?

13.3-2

Show the red-black trees that result after successively inserting the keys 41, 38, 31, 12, 19, 8 into an initially empty red-black tree.

13.3-3

Suppose that the black-height of each of the subtrees $\alpha, \beta, \gamma, \delta, \epsilon$ in Figures 13.5 and 13.6 is k . Label each node in each figure with its black-height to verify that the indicated transformation preserves property 5.

13.3-4

Professor Teach is concerned that RB-INSERT-FIXUP might set $T.nil.color$ to RED, in which case the test in line 1 would not cause the loop to terminate when z is the root. Show that the professor's concern is unfounded by arguing that RB-INSERT-FIXUP never sets $T.nil.color$ to RED.

13.3-5

Consider a red-black tree formed by inserting n nodes with RB-INSERT. Argue that if $n > 1$, the tree has at least one red node.

13.3-6

Suggest how to implement RB-INSERT efficiently if the representation for red-black trees includes no storage for parent pointers.

13.4 Deletion

Like the other basic operations on an n -node red-black tree, deletion of a node takes $O(\lg n)$ time. Deleting a node from a red-black tree is more complicated than inserting a node.

The procedure for deleting a node from a red-black tree is based on the TREE-DELETE procedure on page 325. First, we need to customize the TRANSPLANT subroutine on page 324 that TREE-DELETE calls so that it applies to a red-black tree. Like TRANSPLANT, the new procedure RB-TRANSPLANT replaces the subtree rooted at node u by the subtree rooted at node v . The RB-TRANSPLANT procedure differs from TRANSPLANT in two ways. First, line 1 references the sentinel $T.nil$ instead of NIL . Second, the assignment to $v.p$ in line 6 occurs unconditionally: the procedure can assign to $v.p$ even if v points to the sentinel. We'll take advantage of the ability to assign to $v.p$ when $v = T.nil$.

RB-TRANSPLANT(T, u, v)

```
1 if  $u.p == T.nil$ 
2    $T.root = v$ 
3 elseif  $u == u.p.left$ 
4    $u.p.left = v$ 
5 else  $u.p.right = v$ 
6  $v.p = u.p$ 
```

The procedure RB-DELETE on the next page is like the TREE-DELETE procedure, but with additional lines of pseudocode. The additional lines deal with nodes x and y that may be involved in violations of the red-black properties. When the node z being deleted has at most one child, then y will be z . When z has two children, then, as in TREE-DELETE, y will be z 's successor, which has no left child and moves into z 's position in the tree. Additionally, y takes on z 's color. In either case, node y has at most one child: node x , which takes y 's place in the tree. (Node x will be the sentinel $T.nil$ if y has no children.) Since node y will be either removed from the tree or moved within the tree, the